Национальный исследовательский Томский государственный университет

IDO

Основы языка программирования Python

Николаев Илья Игоревич

Старший преподаватель
Младший научный сотрудник НИЛ «БЭМС РЭС»

Основные разделы курса

Основы языка программирования Python

Оцениваемые практики – 6 шт (30 баллов)
Тест №1 (10 баллов)

Модули и пакеты

Тест №2 (10 баллов)

Выполнение запросов к ресурсам сети

Оцениваемые практики – 1 шт (5 баллов)
Тест №3 (10 баллов)

Объектно-ориентированное программирование

Оцениваемые практики – 1 шт (5 баллов)
Тест №4 (10 баллов)

Создание графического интерфейса

Оцениваемые практики – 1 шт (5 баллов)

Анализ данных: NumPy, Matplotlib, Pandas

Оцениваемые практики – 1 шт (5 баллов)
Тест №5 (10 баллов)

Unit тесты. Регулярные выражения

Итого: 100 баллов

Програм мирование

Процесс создания компьютерных программ путём написания набора инструкций (кода) на специальном языке программирования, который компьютер может понять и выполнить (способ "разговаривать" с компьютером).

```
BB 11 01 B9 0D 00 B4 0E
8A 07 43 CD 10 E2 F9 CD
20 48 65 6C 6C 6F 2C 20
77 6F 72 6C 64 21
```

Python

Это высокоуровневый язык программирования, известный своей простотой и читаемостью.

Простой синтаксис - код читается почти как обычный текст.

Широкая сфера применения - создание сайтов, игр, анализ данных.

Большое сообщество и множество библиотек - легче получить поддержку и найти готовые решения.

```
print("Hello,world!")
```

Програм мирование

Процесс создания компьютерных программ путём написания набора инструкций (кода) на специальном языке программирования, которые компьютер может понять и выполнить (способ "разговаривать" с компьютером).

```
name = input(“Введите своё имя ”)
print(“Привет,” + name)
```

Как запустить код на Python

Компиляция

Программа полностью переводится в машинный (или промежуточный) код перед выполнением.
(C++, Rust)

Интерпретация

Программа выполняется построчно, переводя код в машинные инструкции на лету.
(Python, JavaScript)

Python - **интерпретируемый язык**, но на самом деле его код сначала компилируется в байт-код, а уже скомпилированный байт-код выполняется **интерпретатором** (то есть переводится в машинный код для процессора).

CPython

Основные элементы синтаксиса

набор правил, определяющих как пишутся программы на Python

Комментарии

Однострочные комментарии

Начинающиеся с символа # и игнорируются интерпретатором:

```
# Это однострочный комментарий
```

Многострочные комментарии

Для многострочных комментариев используются тройные кавычки:

```
"""
Это многострочный
комментарий.
Он может занимать
несколько строк.
Часто используется для
docstrings.
"""
```

Практическое применение

```
def calculate_bmi(weight, height):
"""
Вычисляет индекс массы тела.
Args:
        weight (float): Вес в килограммах
        height(float): Рост в метрах
Returns:
        float: Значение индекса массы тела
"""
        return weight / (height ** 2)
```

Основные элементы синтаксиса

набор правил, определяющих как пишутся программы на Python

https://peps.python.org/pep-0008/
https://pythonworld.ru/osnovy/pep-8-rukovodstvo-po-napisaniyu-koda-na-python.html
https://docs.python.org/3/tutorial/introduction.html

Отступы

Python

```
# ✅ Правильно (4 пробела на уровень)
# Выровнено по открывающему разделителю
foo = long_function_name(var_one, var_two,
                         var_three, var_four)

# Больше отступов включено для отличения его от остальных
def long_function_name(
        var_one, var_two, var_three,
        var_four):
    print(var_one)
```

```
# ❌ Неправильно (табы или 2 пробела)
# Аргументы на первой линии запрещены, если не используется вертикальное
выравнивание
foo = long_function_name(var_one, var_two,
    var_three, var_four)

# Больше отступов требуется, для отличения его от остальных
def long_function_name(
    var_one, var_two, var_three,
    var_four):
    print(var_one)
```

Переменные

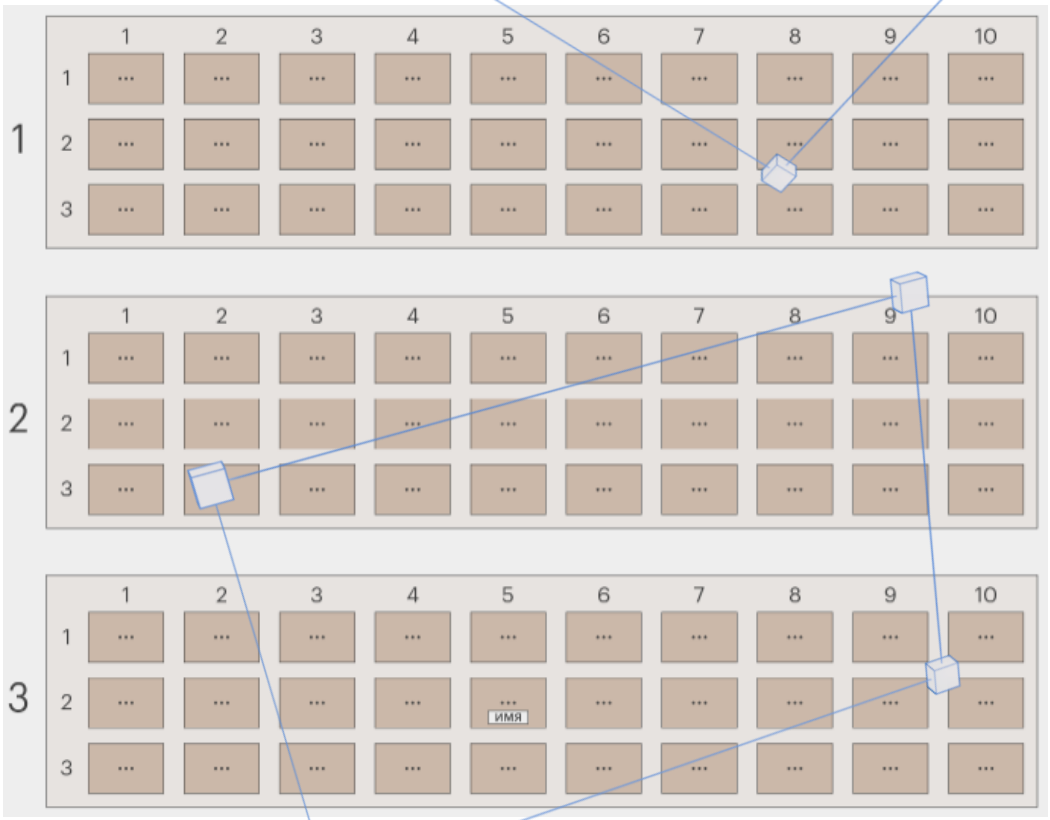
Область памяти компьютера, у которой есть имя. Структурно она состоит из трёх частей

```python
x = 4
name = 'Виктория'
coin_flipping_result = ['орёл', 'решка', 'решка', 'орёл']
```

Имя, или идентификатор — это название, придуманное программистом, чтобы обращаться к переменной. В примерах выше это **x**, **name** и **coin_flipping_result**.

Значение — это информация, которая хранится в памяти компьютера и с которой работает программа. В примерах выше это 4, 'Виктория' и ['орёл', 'решка', 'решка', 'орёл']. Они всегда принадлежат к какому-либо.

Адрес — это номер ячейки памяти, в которой хранится значение переменной.



Переменные

Правила для переменных в Python:

- Имя переменной должно начинаться с буквы (`test`) или символа подчёркивания (`_test`);
- Имя переменной не может начинаться с цифры;
- Имя переменной может содержать только буквенно-цифровые символы и подчёркивание (Az, 0-9 и _);
- Имена переменных чувствительны к регистру (`test`, `Test` и `TEST` — три разные переменные).
- Имя переменной не должно совпадать с зарезервированным ключевым словом.

```python
import keyword
print(keyword.kwlist)
```

```
['False', 'None', 'True', 'and', 'as', 'assert', 'async', 'await',
'break', 'class', 'continue', 'def', 'del', 'elif', 'else', 'except
', 'finally', 'for', 'from', 'global', 'if', 'import', 'in', 'is',
'lambda', 'nonlocal', 'not', 'or', 'pass', 'raise', 'return', 'try'
, 'while', 'with', 'yield']
```

Если нарушить правило именования, получим SyntaxError:

```python
3r = 10
print(3r)
```

```
  File "test.py", line 1
    3r = 10
     ^
SyntaxError: invalid syntax
```

Переменные

В Python тип переменной определяется автоматически и может изменяться:

```python
x = 10          # x — целое число (int)
x = "десять"    # теперь x — строка (str)
x = [1, 2, 3]   # теперь x — список (list)
```

Узнать текущий тип переменной можно с помощью функции **type()** :

```python
print(type(x))  # Выведет <class 'list'>
```

Присваивание

Простое присваивание

```python
# Одновременное присваивание
x, y, z = 1, 2, 3

# Обмен значений
a, b = b, a  # Элегантный обмен без временной переменной

# Присваивание одного значения нескольким переменным
x = y = z = 0
```

Множественное присваивание

```python
# Одновременное присваивание
x, y, z = 1, 2, 3

# Обмен значений
a, b = b, a  # Элегантный обмен без временной переменной

# Присваивание одного значения нескольким переменным
x = y = z = 0
```

Типы данных

int - представляют целые числа без дробной части:

a = 10

float - представляют числа с дробной частью:

a = 9.8

complex - представляют комплексные числа с действительной и мнимой частями:

c = 2 + 3j

Числа

Арифметические операторы:

1. \+ Сложение
2. \- Вычитание
3. * Умножение
4. / Деление
5. % Остаток от деления
6. ** Возведение в степень
7. // Целочисленное деление

a = 10, b = 2:

1. a + b = 12
2. a – b = 8
3. a * b = 20
4. a / b = 5
5. a % b = 0
6. a ** b = 100
7. a // b = 5

Типы данных

Строки (string) — это последовательности символов.

Они используются для хранения текстовой информации и заключаются в одиночные '...' или двойные кавычки "...".

https://pythonworld.ru/tipy-dannyx-v-python/stroki-funkcii-i-metody-strok.html

Строки

Операции со строками:

- **Конкатенация (соединение/сложение)**:

```
str1 = "Hello"
str2 = "World"
result = str1 + " " + str2  # "Hello World"
```

- **Повторение**:

```
str3 = "Ha" * 3 # "HaHaHa"
```

- **Доступ по индексу**:

```
first_char = str3[0] # "H"
```

- **Длина строки**:

```
len(str1) # 5
```

- **Извлечение среза**:

```
substring = str1[1:4]  # "ell"
```

Типы данных

Строки (string) — это последовательности символов.

Они используются для хранения текстовой информации и заключаются в одиночные '...' или двойные кавычки "...".

https://pythonworld.ru/tipy-dannyx-v-python/stroki-funkcii-i-metody-strok.html

Строки

Операции со строками:

- **Разбиение строки по разделителю s.split()**:

```
str1 = "Hello, Adam"
print(str1.split()) # ["Hello", "Adam"]
```

- **Удаление символов в начале конце строки s.strip()**:

```
str1 = "Hello, Adam?!"
print(str1.strip()) # Hello, Adam?!
```

Форматирование строк:

f-строки (Python 3.6+):

```
age = 25
info = f"{name} is {age} years old."
# "Alice is 25 years old."
```

Типы данных

Логический тип данных (**bool** или **булевый тип**) — это примитивный тип данных, который принимает 2 значения — **истина** или **ложь**.

True и **False** являются экземплярами класса **bool** который в свою очередь является подклассом **int** Поэтому **True** и **False** в Python ведут себя как числа **1** и **0**.

Булевы значения

В Python имеется самостоятельный логический тип **bool**, с двумя предопределёнными значениями:

- **True** — истина;
- **False** — ложь.

- непустая строка (в том числе если это один или несколько пробелов);
- ненулевое число (в том числе меньшее единицы, например -5);
- непустой список/кортеж (даже если он содержит один пустой элемент, например пустой кортеж);
- функция.

- пустая строка;
- нулевое число;
- пустой список/кортеж.

Знакомство с операторами

Операторы позволяют выполнять различные операции над данными

Операторы сравнения:

== Равно

!= Не равно

> Больше

< Меньше

>= Больше или равно

<= Меньше или равно

Операторы сравнения

a = 5, b = 10

```
print(a == b)  # False

print(a != b)   # True

print(a > b)    # False

print(a < b)    # True

print(a >= b)  # False

print(a <+ b)  # True
```

Знакомство с операторами

Операторы позволяют выполнять различные операции над данными

Операторы присваивания:

= присваивание

+= Сложение с присваиванием

-= Вычитание с присваиванием

*=, /=, **=, %=, //= Аналогично для других арифметических операций

Операторы присваивания

```
num = 5

num += 3  # num = num + 3

num -= 3  # num = num - 3

num *= 3  # num = num * 3

num /= 3  # num = num / 3

num **= 3  # num = num ** 3
```

Структуры данных

Списки (list) - упорядоченные изменяемые коллекции объектов произвольных типов.

- Изменяемость
- НЕ упорядоченность
- Элементами списка могут быть объекты разных типов данных

Создание списка:

```
s = [ ] # Пустой список
s = list() # Пустой список

list("two") # [`t`, `w`, `o`]
```

+ все операции строк (доступ по индексу, длина, среза)

Списки

Метод	Что делает
list.append(x)	Добавляет элемент в конец списка
list.extend(L)	Расширяет список list, добавляя в конец все элементы списка L
list.insert(i, x)	Вставляет на i-ый элемент значение x
list.remove(x)	Удаляет первый элемент в списке, имеющий значение x. ValueError, если такого элемента не существует
list.pop([i])	Удаляет i-ый элемент и возвращает его. Если индекс не указан, удаляется последний элемент
list.index(x, [start [, end]])	Возвращает положение первого элемента со значением x (при этом поиск ведется от start до end)
list.count(x)	Возвращает количество элементов со значением x
list.sort([key=функция])	Сортирует список на основе функции
list.reverse()	Разворачивает список
list.copy()	Поверхностная копия списка
list.clear()	Очищает список

Структуры данных

Кортежи (tuple) — это неизменяемый тип данных в Python, который используется для хранения упорядоченной последовательности элементов.

- Неизменяемость
- Упорядоченность
- Элементами кортежа могут быть объекты разных типов данных

Создание кортежа:

```
s = () # Пустой кортеж
s = tuple() # Пустой кортеж
s = 1, 2, 3 # (1, 2, 3)
```

Кортежи

Когда использовать?

- Защита элементов от изменений
- Экономия памяти

Например если нужно сохранить координаты какой-либо точки, дни недели и т.д.

Основные операции:

- Доступ по индексу
- Посчитать длину
- Соединение
- Повторение

tpl1=(1,2,3), tpl2=(4,5,6)

- tpl1[0]
- len(tpl1)
- new_tup = tpl1 + tpl2
- xtpl = tpl1*2

Основные методы:

- **count()** - количество раз, когда элемент встречается в кортеже
- **index()** - индекс первого вхождения указанного элемента. То есть первый раз, когда элемент встречается в последовательности.

```
my_tuple = (1, 2, 3, 2, 4, 2)
print(my_tuple.count(2)) #3

print(my_tuple.index(3)) #2
```

Структуры данных

Кортежи (tuple) — это неизменяемый тип данных в Python, который используется для хранения упорядоченной последовательности элементов.

- Неизменяемость
- Упорядоченность
- Элементами кортежа могут быть объекты разных типов данных

Создание кортежа:
s = () # Пустой кортеж
s = tuple() # Пустой кортеж
s = 1, 2, 3 # (1, 2, 3)

Кортежи

Как сортировать кортежи?

Напрямую кортежи сортировать нельзя, так как это действие также является изменением. Но есть другой способ отсортировать данные: **преобразовать кортеж в список, отсортировать его, а затем, при необходимости, вернуть обратно в кортеж.**

1. Создаем кортеж **my_tuple**:
2. Преобразуем кортеж в список с помощью **list()**.
3. Сортируем получившийся список (по умолчанию от меньшего к большему) и получаем новый отсортированный список.
4. Преобразовываем список обратно в кортеж

```
my_tuple = (4, 1, 3, 2)

my_list = list(my_tuple)

sorted_list = sorted(my_list)

new_tuple = tuple(sorted_list)
```

Распаковка кортежа - это способ присвоить его элементы отдельным переменным. Это удобно, когда нужно быстро извлечь значения из кортежа.

```
my_tuple = ("Иван", 25)
name, age = my_tuple
print(name) # Иван
print(age) # 25
```

Структуры данных

Множества

Множество (set) — это изменяемый набор уникальных и неупорядоченных элементов.

- Уникальность
- Неупорядоченность
- Изменяемость

Создание множества:

```
s = {} # Пустой кортеж
s = set() # Пустой кортеж
s = frozenset() # неизменяемое множество
```

Метод	Что делает
add()	Добавляет во множество один объект
update()	позволяет добавить сразу несколько элементов. Аргументом может быть список, строка или другой итерируемый объект.
union()	Позволяет объединять множества друг с другом. Если в двух наборах есть повторяющиеся объекты, в их «гибриде» останется только один.
remove ()	Удаляет элемент из множества. Если такого элемента в наборе не окажется, интерпретатор выдаст исключение.
discard()	Удаляет элемент из множества. Если элемент не найден, исключения не выпадет.
clear()	Если же вы хотите полностью очистить множество, используйте метод
pop()	Удаляет случайный компонент множества.

Структуры данных

Множество (set) — это изменяемый набор уникальных и неупорядоченных элементов.

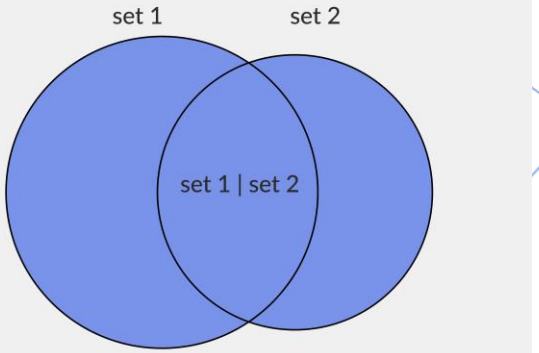
- Уникальность
- Неупорядоченность
- Изменяемость

Создание множества:
s = {} # Пустой кортеж
s = set() # Пустой кортеж
s = frozenset() # неизменяемое множество

Операции над математическими множествами

1. Объединение (union или |)

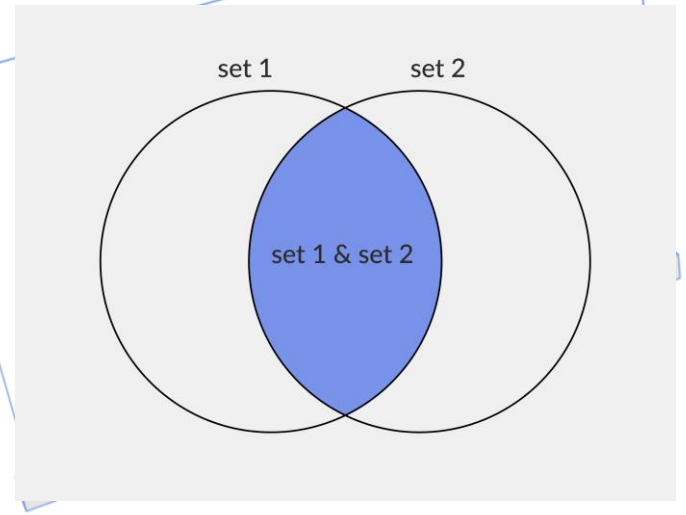
```
set1 = {1, 2, 3}
set2 = {3, 4, 5}
set3 = set1 | set2
print(set3) # Вывод:
{1, 2, 3, 4, 5}
```



2. Пересечение (intersection или &)

```
set1 = {1, 2, 3, 4}
set2 = {3, 4, 5, 6}
set3 = set1.intersection(set2)
print(set3)  # {3, 4}

set3 = set1 & set2
print(set3)  # {3, 4}
```



Структуры данных

Множество (set) — это изменяемый набор уникальных и неупорядоченных элементов.

- Уникальность
- Неупорядоченность
- Изменяемость

Создание множества:

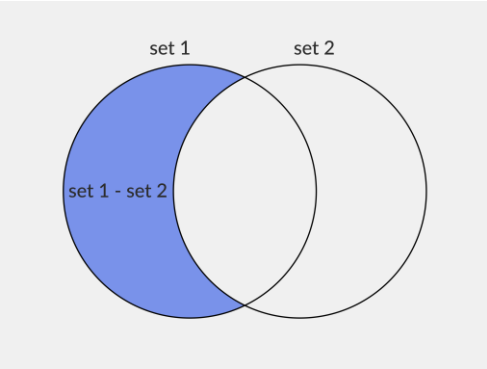
s = {} # Пустой кортеж

s = set() # Пустой кортеж

s = frozenset() # неизменяемое множество

Операции над математическими множествами

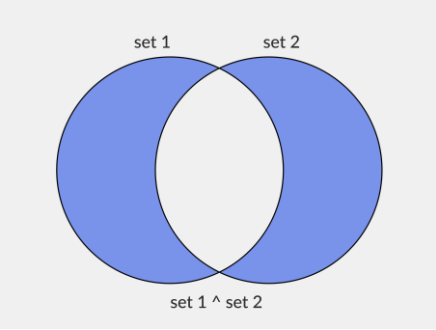
3. Разность (difference или -)




```
set1 = {"apple", "banana", "cherry"}
set2 = {"banana", "orange", "mango"}
set3 = set1 - set2
print(set3) # Выведет {'cherry', 'apple'}

set3 = set1.difference(set2)
print(set3) # Выведет {'cherry', 'apple'}
```

4. Симметрическая разность (symmetric_difference() или ^)




```
a = {"apple", "banana", "cherry"}
b = {"banana", "orange", "mango"}
c = a ^ b
print(c)
# Выведет {'apple', 'cherry','orange','mango'}
c = a.symmetric_difference(b)
print(c)
# Выведет {'apple', 'cherry','orange','mango'}
```

Структуры данных

Словари в Python - неупорядоченные коллекции произвольных объектов с доступом по ключу.

Создание словаря:
d = {}
d = {“ dict” : “1 ”}

Словари

Метод	Что делает
clear()	Удаляет все элементы из словаря
copy()	Создаёт копию словаря
update()	Добавляет в словарь одну или несколько пар «ключ — значение»
pop()	Удаляет элемент из словаря по ключу
keys()	Возвращает список ключей из словаря
values()	Возвращает список значений из словаря
items()	Возвращает все пары «ключ — значение»
popitem()	Удаляет последнюю добавленную пару «ключ — значение»
get()	Возвращает значение из словаря по ключу — или None, если такого ключа не существует. Можно указать своё значение в качестве второго аргумента
setdefault()	Возвращает значение указанного ключа. Если ключа не существует, создаёт его и добавляет в словарь со значением None или тем, что вы добавите в качестве второго аргумента

Условные операторы

```
if условие:
        Блок_кода
```

```
x = 10
if x > 5:
        print('x больше 5’)
```

```
if условие:
        Блок_кода_1
else:
        Блок_кода_2
```

```
x = 10
if x > 5:       # Если x больше 5, выводим, что больше
        print('x больше 5’)
else:           # Если меньше, выводим, что меньше
        print('x меньше 5')
```

```
if условие1:
        блок_кода1
elif условие2:
        блок_кода2
elif условие3:
        блок_кода3
else:
        блок_кодаN
```

```
x = 7
if x > 10:
        print("x больше 10")
elif x > 5:
        print("x больше 5, но меньше
или равно 10")
elif x > 0:
        print("x больше 0, но меньше
или равно 5")
else:
        print("x меньше или равно 0")
```

Циклы



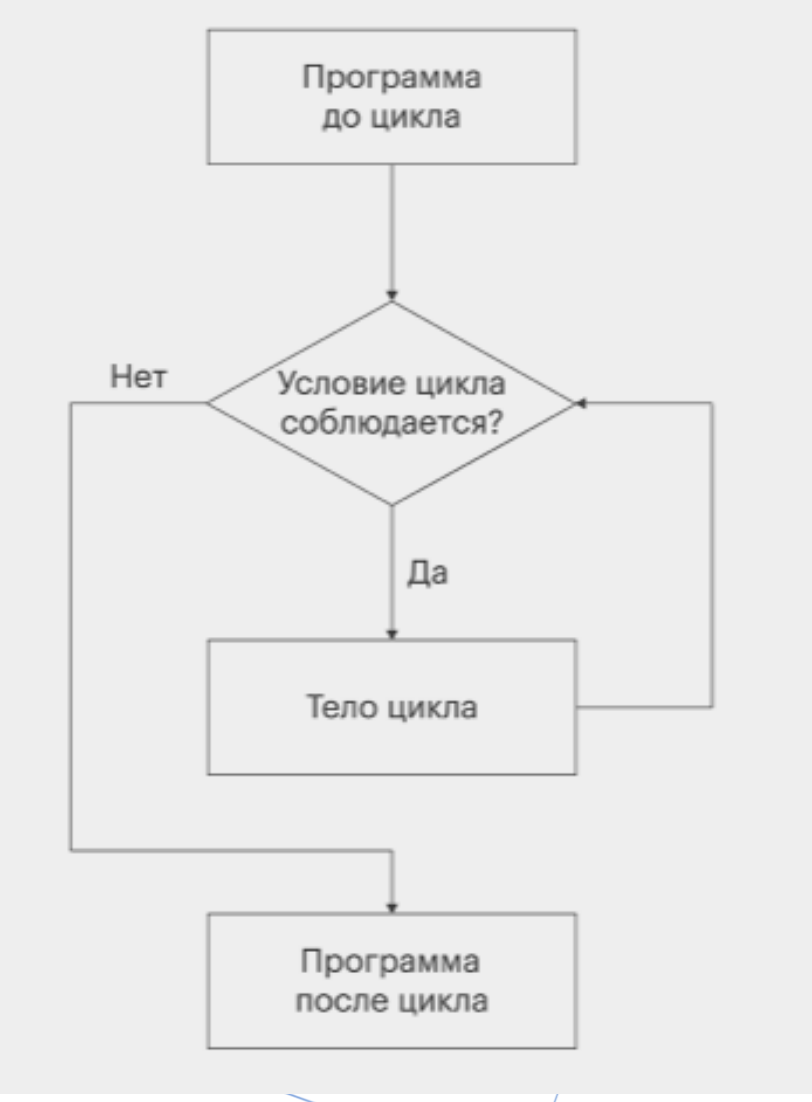
while — наиболее простой и понятный вид цикла.
Его ещё называют циклом с предусловием.

```
x = 1
while x < 5:
    print(x)
    x += 1  # Означает то же самое, что x = x + 1

# Этот код будет выполняться бесконечно
x = 1
while x < 5:
      print(x)

while True:
      pass  # pass — оператор-заглушка
```

Циклы



Цикл **for** программисты используют куда чаще, чем while.
Для него мы устанавливаем не условие в чистом виде, а некий массив данных: **список**, **кортеж**, **строку**, **словарь**, **диапазон** или любой другой итерируемый объект.

```
num_list = [14, 101, -7, 0]
for number in num_list:
    print(number)
```

Когда нужно применить **for** к числовому промежутку, его можно задать диапазоном. Для этого используют функцию **range()**. В неё можно передать от одного до трёх аргументов.

```
for i in range(3):
    print(i)
```

```
for i in range(20, 25):
    print(i)
```

```
for i in range(10, 20, 5):
    print(i)
```

Прерывание цикла

break – необходим для досрочного принудительного завершения цикла

```
string = 'Hi, loop!'
for i in string:
    if i == ',':
        break
    print(i)
>>> H
>>> i
```

```
string = 'Hi loop!'
for i in string:
    if i == ',':
        break
    print(i)
>>> H
>>> i
>>>
>>> l
>>> o
>>> o
>>> p
>>> !
```

Пропуск части цикла

continue – необходим для начала следующего шага цикла, пропустив часть строк в его теле

```
for i in range(1, 10):
    if i%2 == 0 or i%3 == 0:
        continue
    print(i)
>>> 1
>>> 5
>>> 7
```

ZIP

zip – позволяет пройтись одновременно по нескольким итерируемым объектам (спискам и др.)

```
a = [“пн”, “вт”, “ср”, “чт”, “пт”]
b = [1, 2, 3, 4, 5]
for i, j in zip(a, b):
        print(j, i)
>>> 1 пн
>>> 2 вт
>>> 3 ср
>>> 4 чт
>>> 5 пт
```

ENUMERATE

enumerate() – инструмент для циклов, создающий пары, состоящие из счётчика и элементов итерируемого объекта

```
lst = ['a', 'b', 'c']
for i, v in enumerate(lst):
    print(i, v)
>>> 0 a
>>> 1 b
>>> 2 c
```

MATCH-CASE

Он позволяет сделать выбор действия при совпадении некоторого субъекта с одним из **шаблонов**.

```
day = 1
match day:
 case 1: print("Понедельник")
 case 2: print("Вторник")
 case 3: print("Среда")
 case 4: print("Четверг")
 case 5: print("Пятница")
 case 6: print("Суббота")
 case 7: print("Воскресенье")
 case _: print("Некорректное значение")
```

Ввод и вывод данных в Python

Функция input

```
name = input('Введите ваше имя:')
print('Ваше имя:'+name)
print(f 'Ваше имя: {name}')
```

Функция print

```
print(value, ..., sep='', end='\n')

print('Привет!', 'Как твои дела?')
```

Национальный исследовательский Томский государственный университет

IDO

Спасибо за внимание!

Готов выслушать Ваши вопросы и ответить на них